

Code Fire Specification and Design v0.2

Table of Contents

1 Code Fire 仕様・設計ドキュメント v0.2

この文書は、Code Fire の思想背景、外部仕様、内部実装設計、MVP 計画を統合したものです。

2 00. 思想背景

2.1 0.1 問題意識

のバージョン管理は、主にファイルの履を管理してきた。Git はその代表であり、非常に力な分散バージョン管理システムである。しかし、Git の基本的な関心は、ファイル集合のスナップショット、差分、ブランチ、マジである。

一方、現代のソフトウェア開で本に問題になるのは、ファイルが保存されているかどうかだけではない。

問題は、次のような成果物同士の意味的な乖離である。

仕様書と設計書がずれる
設計書とコードがずれる
コードとテストがずれる
仕様書とテストがずれる
API 仕様と実装がずれる
ADR と現在の実装判断がずれる

AI コーディングが普及すると、この問題はさらに大きくなる。AI は大量のコード変更を高速に実行できるが、その速度は仕様書、設計書、テスト、トレーサビリティの更新速度を超えやすい。結果として、コードだけが先に進み、周辺成果物が置き去りになる。

Code Fire は、この状況に対して次の原則を置く。

仕様・設計・コード・テストが整合した状態だけを正式履歴に残す。

2.2 0.2 Git 的自由からの離脱

Git の強さは自由度にある。

好きな時点で commit できる
WIP commit を作れる
stash できる
partial commit できる
rebase できる
force push できる
pull で取得と統合をまとめて行える

しかし、Code Fire の目的から見ると、この自由度は副作用を持つ。

不整合態を commit できるなら、履歴は「整合済み状態の列」ではなくなる。WIP commit や checkpoint が使えるなら、不整合状態が管理対象として保存される。stash があれば、未整合の変更を横に置いて後から戻せる。partial commit があれば、仕様・設計・コード・テストが分断される。

Code Fire は、これらを機能不足としてではなく、意図的に提供しない。

Code Fire は、開発者に自由な履歴操作を与えるツールではない。開発者と AI に対して、整合する小さな変更単位を作ることを強制するツールである。

2.3 0.3 AI 時代の開発規律

AI コーディング時代には、人間が一つひとつのファイル変更を把握し続けることは難しくなる。その代わりに、AI と人間が共有できる作業単位を定義する必要がある。

Code Fire では、その作業単位を fire として扱う。

fire は不整合そのものではなく、未解消の確認責任である。

例：

CODE-SessionPolicy が変更された

↓

DES-AUTH-001 を確認する必要がある

REQ-AUTH-001 を確認する必要がある

TEST-session-expiration を確認する必要がある

この確認責任がすべて解消されるまで、commit はできない。

AI には、「リポジトリをいい感じに直せ」と指示するのではなく、次のように指示できる。

未解消 fire をすべて解消し、commit 可能状態にせよ。

これは AI 開発において極めて重要である。AI の作業目標が、曖昧な自然言語タスクから、明示的な整合回復タスクになるからである。

2.4 0.4 履の意味をえる

Git の commit は、おおむねファイルスナップショットである。

Code Fire の commit は異なる。

Code Fire の commit は、以下を含む sealed state である。

成果物ファイルの内容

Artifact Atom 索引

Trace Graph

fire 解消記録

検証結果

policy

整合性証明

つまり、Code Fire の commit は保存ではなく、封印である。

Code Fire commit = consistent system state seal

このため、正式履歴上の任意の commit は、少なくとも Code Fire policy 上は整合済みである。

2.5 0.5 Code Fire の価値

Code Fire が提供する価値は、履歴の自由度ではない。

Code Fire の価値は、次の保証である。

任意の commit を開いても、未解消 fire は存在しない

任意の commit は、その時点の policy に基づく整合性証明を持つ

任意の commit には、なぜ変更不要と判断したかの証跡が残る

server や merge request には整合済み状態だけが流通する

この保証は、AI コーディングの高速化と引き換えに失われがちな意味的整合性を、開発フローの中核に戻す。

3 01. プロダクト定義

3.1 1.1 一文定義

Code Fire は、仕様・設計・コード・テスト・トレーサビリティを単一のシステムグラフとして管理し、未解消の確認責任が存在する状態を正式履歴に commit することを禁止する、整合性ファーストの仕様駆動 VCS である。

短く言えば、次である。

Code Fire は、整合済みシステム状態だけを commit できる、AI 時代の仕様駆動バージョン管理システムである。

3.2 1.2 プロダクト目的

Code Fire の目的は、以下の状態を防ぐことである。

コードだけが変わったが、仕様は未確認
仕様は変わったが、テストは未更新
設計判断は変わったが、ADR が古い
API 挙動は変わったが、OpenAPI 定義が古い
テストは変わったが、要求仕様との関係が追えない

Code Fire は、成果物間の関係を Trace Graph として管理し、変更時に関連成果物へ fire を発火させる。fire がすべて extinguish されるまで、commit は拒否される。

3.3 1.3 非 Git 方針

Code Fire は Git と共存しない。Git を内包するわけでも、Git repository を裏側に使うわけでもない。

以下は提供しない。

checkout
current branch
staging area
partial commit
stash
checkpoint
pull
fetch
rebase
force upload
remote tracking branch
WIP commit

この非提供は制約ではなく、プロダクト思想である。

3.4 1.4 提供する中核

ユザは以下である。

1. sealed branch を clone する
2. branch を open し、実ディレクトリとして展開する
3. 仕様・設計・コード・テストを編集する
4. scan により変更 Atom を検出する
5. Trace Graph に基づき fire が立つ
6. fire を extinguish する
7. verify する
8. consistent なら commit できる

- 9. 必要なら upload または merge request する
- 10. close すると実ディレクトリが消える

3.5 1.5 基本保証

Code Fire は以下を保証する。

branch head は常に sealed commit を指す
sealed commit は常に consistent certificate を持つ
server に upload できるのは sealed branch のみ
merge request にできるのは sealed branch のみ
open-burning 状態は正式履歴に入らない
open-burning 状態は server に流通しない

3.6 1.6 Code Fire が保証しないこと

Code Fire は、世界に対する絶対的な正しさを保証しない。

Code Fire が保証するのは、次である。

対象 commit が、その時点の Code Fire policy 上で、未解消の確認責任を持たず、必要なら通っていること。

したがって、policy が不十分なら、Code Fire 上で consistent でも実際のビジネス要求とずれる可能性はある。Code Fire は真理判定機ではなく、確認責任の管理システムである。

4 02. 中核概念

4.1 2.1 Repository

Repository は、Code Fire が管理するプロジェクト全体の保管庫である。

Repository は以下を持つ。

sealed commit 群
branch 群
object store
policy
open directory 管理情報
server 接情報
cache

Repository に current branch は存在しない。

4.2 2.2 Branch

Code Fire の branch は、Git branch ではない。

Code Fire branch は、整合済み commit だけからなる開発線である。

C1 consistent -> C2 consistent -> C3 consistent

branch head は常に sealed commit を指す。

4.3 2.3 Open Directory

branch は、open すると実ディレクトリとして展開される。

```
codefire open main ./main
```

```
codefire open feature-login ./feature-login
```

これにより、複数 branch を同時に開ける。

```
project/  
  .codefire/  
    main/  
    feature-login/
```

Git の checkout のように、1つの作業ディレクトリを切り替えることはしない。

4.4 2.4 Close

close は open された branch directory を削除する。

```
codefire close feature-login
```

clean でない branch は通常 close できない。

```
codefire close feature-login --discard
```

を使うと、未 commit 作業を破棄して close する。

4.5 2.5 Clone

Code Fire の clone は、sealed branch head から新しい branch を作る操作である。

```
codefire clone main feature-login
```

open directory の未 commit 状態をコピーする操作ではない。

4.6 2.6 Commit

Code Fire の commit は、整合済み状態の封印である。

commit には以下が含まれる。

content manifest
artifact index
atom index
trace graph
fire delta
resolution records
verification result
policy
consistency certificate

未解消 fire が残っている状態では commit できない。

4.7 2.7 Artifact

Artifact は、Code Fire が管理する成果物である。

例：

仕様書
設計書
ADR
API 仕様
DB スキーマ
コード
テスト
運用 Runbook

4.8 2.8 Artifact Atom

Artifact Atom は、成果物内の意味的な最小管理単位である。

例：

REQ-AUTH-001
DES-AUTH-001
CODE-SessionPolicy
TEST-session-expiration
API-login

Code Fire ではファイル単位ではなく Atom 単で変影響を管理する。

4.9 2.9 Trace Link

Trace Link は Artifact Atom 同士の係である。

例：

REQ-AUTH-001 refined_by DES-AUTH-001
DES-AUTH-001 implemented_by CODE-SessionPolicy
REQ-AUTH-001 verified_by TEST-session-expiration

トレサビリティ表は Trace Link から生成される派生ビューであり、手編集する一次情報ではない。

4.10 2.10 Fire

fire は未解消の確認責任である。

CODE-SessionPolicy changed

-> DES-AUTH-001 の確認が必要

-> REQ-AUTH-001 の確認が必要

-> TEST-session-expiration の確認が必要

fire は commit blocker である。

4.11 2.11 Extinguish

extinguish は fire を解消する操作である。

解消方法：

changed

verified

no-change-required

invalid-fire

no-change-required でも理由が必要である。

5 03. 開発フロー仕様

5.1 3.1 標準フロ

Code Fire の標準フローは以下である。

remote/local の sealed branch を clone

↓

branch を open して実ディレクトリ化

↓

編集

↓

scan

↓

fire 発火
↓
extinguish
↓
verify
↓
commit
↓
upload または merge request
↓
close

5.2 3.2 例 : feature branch を作る

```
codefire clone main feature-login  
codefire open feature-login ./feature-login
```

5.3 3.3 例 : 変更して fire を立てる

```
cd feature-login  
# src/auth.py などを編集  
codefire scan
```

出力例 :

Branch: feature-login
State: open-burning
Commit: blocked

Changed atoms:
CODE-SessionPolicy

Open fires:
FIRE-001 CODE-SessionPolicy -> DES-AUTH-001
FIRE-002 CODE-SessionPolicy -> REQ-AUTH-001
FIRE-003 CODE-SessionPolicy -> TEST-session-expiration

5.4 3.4 fire 解消

設計書を変更した場合 :

```
codefire extinguish FIRE-001 \  
--resolution changed \  
--evidence docs/design/auth.md#DES-AUTH-001
```

仕様変更不要の場合 :

```
codefire extinguish FIRE-002 \  
--resolution no-change-required \  
--rationale "外部仕は化しておらず、部の整理のみであるため"
```

テストを更新した場合：

```
codefire extinguish FIRE-003 \  
--resolution changed \  
--evidence tests/test_auth.py#TEST-session-expiration
```

5.5 3.5 verify

codefire verify

出力例：

Verification passed.

Open fires: 0

Missing required links: 0

Stale resolutions: 0

Duplicate atom ids: 0

State: open-consistent

Commit: allowed

5.6 3.6 commit

```
codefire commit -m "Change session expiration policy"
```

出力例：

Sealed commit created.

Commit: CF-COMMIT-8f3a91d0c2

Branch: feature-login

State: open-clean

5.7 3.7 close

codefire close feature-login

実ディレクトリ ./feature-login は削除される。branch 履歴は残る。

5.8 3.8 長い作業の扱い

checkpoint は存在しない。長期間 commit できない作業は、変更単位が大きすぎるというシグナルである。

Code Fire では、大きな変更を以下のように分割することを推奨する。

1. 互換性を保った内部構造変更を整合 commit する
2. 仕様に見える小変更を整合 commit する
3. テストと設計を更新して整合 commit する
4. 古い経路を削除して整合 commit する

AI にも「次の整合可能な最小 commit を作る」ことを指示する。

6 04. CLI 仕様

6.1 4.1 最小コマンドセット

codefire init

codefire branch list

codefire open <branch> <path>

codefire close <branch>

codefire close <branch> --discard

codefire clone <source-branch-or-url> <new-branch>

codefire status

codefire scan

codefire fire <atom-id> --reason <text>

codefire extinguish <fire-id> --resolution <type> [--rationale <text>] [--evidence <ref>]

codefire verify

codefire commit -m <message>

codefire merge <source-branch> --into <target-branch>

codefire upload <branch> <server-url>

codefire request-merge <source-url> <target-url>

codefire list <server-url>

codefire show <branch-or-url>

codefire diff <branch-or-url> <branch-or-url>

codefire request-list <server-url>

codefire discard <branch>

codefire doctor

6.2 4.2 作らないコマンド

checkout
pull
fetch
rebase
stash
checkpoint
add
stage
reset --soft
force-upload

6.3 4.3 open

codefire open main ./main

仕様：

branch head の sealed commit を指定 path に展開する

同一 branch の多重 open は禁止する

target path は存在しないか空でなければならない

.codefire-open を生成する

open registry を更新する

6.4 4.4 close

codefire close feature-login

仕様：

open-clean の場合のみ実ディレクトリを削除する

open-burning または open-consistent の場合は拒否する

破棄して閉じる場合：

codefire close feature-login --discard

6.5 4.5 clone

codefire clone main feature-login

仕様：

source branch の sealed head を target branch の head として設定する

source が open-burning の場合は拒否する

target branch は closed 状態で作成される

remote から取得する場合：

codefire clone cf://server/org/app/main main-latest

remote tracking branch は作らない。

6.6 4.6 scan

codefire scan

仕様：

open directory を index する

Atom 単位で変更を検出する

Trace Graph に基づき fire を生成する

obsolete fire を整理する

branch state を更新する

6.7 4.7 fire

codefire fire REQ-AUTH-001 --reason "仕様と実装が一致していない可能性がある"

仕様：

manual fire を立てる

manual fire は revert で自動消滅しない

extinguish が必要

6.8 4.8 extinguish

codefire extinguish FIRE-001 \

--resolution no-change-required \

--rationale "外部仕様は変化していない"

仕様：

fire を解消する

basis として source/target/link/policy hash を保存する

必要な rationale/evidence がない場合は拒否する

6.9 4.9 verify

codefire verify

仕様：

scan を実行する

policy check を実行する

required verification command を実行する

verification object を active state に保存する

6.10 4.10 commit

codefire commit -m "Implement login handler"

仕様：

scan と verify を再実行する
committable 条件を満たす場合だけ sealed commit を作る
branch head を更新する
active state を reset する
open state を open-clean にする

6.11 4.11 merge

codefire merge feature-login --into main

仕様：

source sealed head を target open directory へ統合する
target は open-clean でなければならない
merge 実行時点では commit を作らない
merge fire を生成し、target を open-burning にする

6.12 4.12 upload

codefire upload feature-login cf://server/alice/app/feature-login

仕様：

sealed branch だけ upload できる
open-burning branch は upload 不可
server branch が進んでいたら拒否する
force upload は存在しない

6.13 4.13 request-merge

codefire request-merge \
cf://server/alice/app/feature-login \
cf://server/org/app/main

仕様：

source sealed commit を target に統合してほしいという申請を作る
source/target とも sealed commit を指す
不整合状態は merge request 不可

7 05. 整合性モデル

7.1 5.1 整合の定義

Code Fire における整合は、絶対的な正しさではない。

Code Fire における整合とは、定義された policy のもとで、未解消の確認責任がなく、必須リンクが存在し、に成功している態である。

7.2 5.2 commit 可能条件

作業状態を (W) とする。

変数の定義：

- (W) : 現在の open directory 上の作業状態
- (F_{open}(W)) : 状態 (W) に存在する未解消 required fire の集合
- (C_{fail}(W)) : 状態 (W) で失敗している 検証 の集合
- (L_{missing}(W)) : 状態 (W) で欠落している必須 Trace Link の集合
- (R_{stale}(W)) : 状態 (W) で古くなった resolution の集合
- (D_{dup}(W)) : 状態 (W) で重複している Atom ID の集合

Code Fire で commit 可能である件は以下である。

$$[\text{committable}(W) = |F_{\text{open}}(W)| = 0 \mid |C_{\text{fail}}(W)| = 0 \mid |L_{\text{missing}}(W)| = 0 \mid |R_{\text{stale}}(W)| = 0 \mid |D_{\text{dup}}(W)| = 0]$$

7.3 5.3 open state

branch は以下の状態を持つ。

closed
open-clean
open-burning
open-consistent

7.3.1 closed

branch は存在するが、実ディレクトリは存在しない。

7.3.2 open-clean

実ディレクトリが存在し、branch head と一致している。

7.3.3 open-burning

実ディレクトリに変更または未解消 fire があり、commit 不可である。

7.3.4 open-consistent

変更はあるが、未解消 fire がなく、検証に成功しており、commit 可能である。

7.4 5.4 状態遷移

closed
| open
↓

```
open-clean
| edit / scan / fire / merge
↓
open-burning
| extinguish / verify
↓
open-consistent
| commit
↓
open-clean
| close
↓
closed
```

7.5 5.5 stale resolution

resolution は、fire を解消した時点の source atom、target atom、trace link、policy に対する判断である。

以下のいずれかが変わった場合、resolution は stale になる。

- source atom hash
- target atom hash
- trace link hash
- policy hash

stale resolution が1つでもある場合、commit 不可である。

7.6 5.6 commit certificate

commit 時には、consistent certificate を生成する。

certificate は以下を含む。

- 入力 root
- open fire 数
- failed check 数
- missing link 数
- stale resolution 数
- duplicate atom id 数
- verification 結果
- policy hash

certificate は、その commit がその policy のもとで整合条件を満たしたことを示す。

8 06. Fire / Extinguish 設計

8.1 6.1 fire の意味

fire は、ある変更または発見された不整合により、関連成果物へ確認責任が発生したことを表す。

fire は不整合そのものではない。確認責任である。

8.2 6.2 fire の発生契機

仕様書の変更

設計書の変更

コードの変更

テストの変更

Trace Link の変更

手動 fire

AI による fire 提案

merge による衝突または意味的衝突候補

policy 違反

8.3 6.3 fire severity

MVP では required のみを扱う。

、以下を追加できる。

required

warning

info

ただし、commit blocker になるのは required fire である。

8.4 6.4 fire 生成

変更 Atom を (s) とする。

変数の定義：

- (s) : 変更された Artifact Atom
- (v) : 確認対象候補 Atom
- (G=(V,E)) : Trace Graph
- (V) : Atom 集合
- (E) : Trace Link 集合
- (P) : fire propagation policy
- (I(s)) : 変更 Atom (s) により確認対象になる Atom 集合

単純化した影響集合は以下である。

$[I(s) = \{ v \mid v \text{ reachable_P}(s, v, G) \}]$

MVP では、変更 Atom に incoming/outgoing で直接接続される Atom へ required fire を立てる。

8.5 6.5 fire 重複排除

同じ scan で同じ fire が増殖してはいけない。

fire key は以下から作る。

source_atom_id

target_atom_id

reason

trace_path_hash

base_commit

同じ fire key の open fire が存在する場合は新規作成しない。

8.6 6.6 obsolete fire

auto fire は、変更が revert された場合に obsolete になる。

current atom hash == base atom hash

-> 更なし

-> その変更由来の auto fire は obsolete

manual fire は自動 obsolete にしない。

8.7 6.7 extinguish 種別

changed

象を更して解消した

verified

検証により問題ないと確認した

no-change-required

確認した結果、更不要と判した

invalid-fire

fire 自が不適切だった

8.8 6.8 no-change-required

no-change-required は非常に重要である。

AI 時代には、「変更しなかった理由」も履歴に残す必要がある。

例：

```
codefire extinguish FIRE-002 \  
--resolution no-change-required \  
--rationale "外部仕様は変化せず、内部処理の整理のみであるため"
```

8.9 6.9 resolution basis

resolution には basis を保存する。

```
source atom hash at resolution  
target atom hash at resolution  
trace link hash at resolution  
policy hash at resolution
```

これにより、後から前提が変わった場合に stale 判定できる。

9 07. ロカル Repository / Branch / Open Directory 設計

9.1 7.1 repository 構造

```
project/  
  .codefire/  
    repo.yaml  
    objects/  
    branches/  
    opened/  
    active/  
    cache/  
    locks/  
    remotes/  
  
  main/  
  feature-login/
```

.codefire/ が正式 repository である。

main/ や feature-login/ は open された実ディレクトリであり、close すると削除される。

9.2 7.2 current branch なし

repository に current branch は存在しない。

NG: この repository は今 main を checkout している

OK: main は ./main に open されている

OK: feature-login は ./feature-login に open されている

9.3 7.3 open registry

open された branch は .codefire/opened/ に登録される。

例：

version: 1

branch:

branch_id: br_01HZXA2AE2RPWZ7M2E

name: feature-login

open:

open_instance_id: open_01HZXA3P99HHH3ZC8Q

path: /workspace/project/feature-login

opened_from_commit: CF-COMMIT-21ab77aa00

current_base_commit: CF-COMMIT-21ab77aa00

active_state_path: /workspace/project/.codefire/active/open_01HZXA3P99HHH3ZC8Q

state:

last_known: open-clean

9.4 7.4 .codefire-open

open directory には識別ファイルを置く。

version: 1

repository:

path: /workspace/project/.codefire

repository_id: repo_01HZX9Q5XJ3M8K7G2T

branch:

name: feature-login

branch_id: br_01HZXA2AE2RPWZ7M2E

opened_from_commit: CF-COMMIT-abc123

open:

open_instance_id: open_01HZXA3P99HHH3ZC8Q

opened_path: /workspace/project/feature-login

Code Fire コマンドは、これをもとに open directory の正当性を検証する。

9.5 7.5 同一 branch の多重 open 禁止

同一 branch を複数 path に open することは禁止する。

理由：

1 branch に複数の burning state が存在すると、どちらが正式な変更元か不明になる
多重 open は checkpoint や stash に近い逃げ道になる
fire/resolution の由来が曖昧になる

9.6 7.6 OS コピされた open directory

ユーザは OS レベルで directory をコピーできる。

```
cp -r feature-login feature-login-backup
```

しかし、コピー先では Code Fire 操作を拒否する。

判定：

.codefire-open は存在する

opened_path が現在 path と違う

または open_instance_id が repository 側 registry と一致しない

これは Code Fire 管理外のバックアップであり、checkpoint ではない。

9.7 7.7 active state

open branch ごとに active state を持つ。

```
.codefire/active/open_xxx/  
state.yaml  
events.jsonl  
fires.yaml  
resolutions.yaml  
verification.yaml  
scan.sqlite
```

active state には作業中ファイル内容を保存しない。

保存してよいもの：

```
fire metadata  
resolution metadata  
scan hash  
verification result  
open identity
```

保存してはいけないもの：

```
作業中ファイルの blob  
復元可能 snapshot  
patch  
diff  
checkpoint  
stash
```

10 08. Object Store 設計

10.1 8.1 基本方針

Code Fire は Git を使わず、独自の content-addressed object store を持つ。

object は immutable である。一度作成した object は更新しない。

10.2 8.2 Object ID

全 object は以下で ID を計算する。

[object_id = sha256(type_tag 0x00 canonical_payload)]

変数の定義：

- (object_id) : object の ID
- (sha256) : SHA-256 hash 関数
- (type_tag) : blob, commit, trace_graph などの object 種別
- () : バイト列の連結
- (canonical_payload) : 順序や空白差分を排除した正規化 payload

10.3 8.3 主要 object

BlobObject

ContentManifestObject

ArtifactObject

AtomIndexObject

TraceGraphObject

FireObject

ResolutionObject

VerificationObject

CommitObject

BranchObject

10.4 8.4 ContentManifestObject

commit 時点の全ファイル一覧を表す。

```
{
  "type": "content_manifest",
  "version": 1,
  "entries": [
    {
      "path": "docs/spec/auth.md",
      "kind": "file",
      "mode": "100644",
```

```
    "blob": "CF-BLOB-222bbb"
  }
]
}
```

.codefire-open は manifest 対象外である。

10.5 8.5 AtomIndexObject

commit 時点の Artifact Atom 索引を表す。

```
{
  "type": "atom_index",
  "version": 1,
  "atoms": [
    {
      "atom_id": "REQ-AUTH-001",
      "kind": "requirement",
      "artifact_path": "docs/spec/auth.md",
      "selector": {
        "type": "markdown_heading",
        "value": "REQ-AUTH-001"
      },
      "content_hash": "sha256:aaa111"
    }
  ]
}
```

10.6 8.6 CommitObject

sealed commit を表す。

```
{
  "type": "commit",
  "version": 1,
  "commit_id": "CF-COMMIT-8f3a91d0c2",
  "parents": ["CF-COMMIT-21ab77aa00"],
  "message": "Change session expiration policy",
  "roots": {
    "content_manifest": "CF-MANIFEST-aaa111",
    "atom_index": "CF-ATOMINDEX-ccc333",
    "trace_graph": "CF-TRACE-ddd444",
    "verification": "CF-VERIFY-fff666",
    "policy": "CF-POLICY-ggg777"
  },
}
```

```
"certificate": {  
  "result": "consistent",  
  "open_required_fires": 0,  
  "failed_checks": 0,  
  "missing_required_links": 0,  
  "stale_resolutions": 0,  
  "duplicate_atom_ids": 0  
}  
}
```

10.7 8.7 object 保存タイミング

未 commit ファイル内容は object store へ保存しない。

scan 時:

Atom hash や fire metadata は active state に保存してよい
作業中ファイル内容は blob 化しない

commit 時:

初めて作業中ファイルを BlobObject として保存する
manifest, atom index, trace graph, verification, commit を保存する

これは checkpoint なしの思想を内部実装でも守るための重要ルールである。

11 09. Index / Trace / Policy 設計

11.1 9.1 versioned project files

open directory 内には以下を置く。

```
codefire.yaml  
codefire.links.yaml  
codefire.policy.yaml
```

これらは通常の成果物として commit 対象である。

11.2 9.2 codefire.yaml

プロジェクト構成を定義する。

```
version: 1  
artifacts:  
  requirements:  
    - path: docs/spec/**/*.*.md
```



```
    kind: requirement_document
designs:
  - path: docs/design/**/*.*md
    kind: design_document
code:
  - path: src/**/*.*py
    kind: python_code
tests:
  - path: tests/**/*.*py
    kind: pytest_test
```

11.3 9.3 Atom ID 規約

推奨 prefix :

REQ- requirement
DES- design
ADR- architecture decision record
API- API operation
DB- database schema element
CODE- code symbol
TEST- test case
OPS- operation/runbook

例 :

REQ-AUTH-001
DES-AUTH-001
CODE-SessionPolicy
TEST-session-expiration

11.4 9.4 Markdown Atom 抽出

Markdown では、見出し先頭の ID を Atom ID とする。

REQ-AUTH-001: セッション有効期限

ユザのセッションは最終操作から 30 分で失効する。

同階層以上の次の見出しまでを Atom 本文とし、content hash を計算する。

11.5 9.5 Python Atom 抽出

MVP では、明示 ID コメントを推奨する。

```
# cf-atom: CODE-SessionPolicy
class SessionPolicy:
```

```
def expires_after_minutes(self) -> int:
    return 30
```

明示 ID がない場合は派生 ID を使う。

CODE:src/auth.py::class:SessionPolicy

11.6 9.6 pytest Atom 抽出

cf-atom: **TEST**-session-expiration

```
def test_session_expiration():
    assert SessionPolicy().expires_after_minutes() == 30
```

11.7 9.7 Trace Graph

Trace Link は codefire.links.yaml に定義する。

```
version: 1
links:
  - from: REQ-AUTH-001
    to: DES-AUTH-001
    type: refined_by
  - from: DES-AUTH-001
    to: CODE-SessionPolicy
    type: implemented_by
  - from: REQ-AUTH-001
    to: TEST-session-expiration
    type: verified_by
```

Trace Graph を以下の有向グラフとして扱う。

[$G = (V, E)$]

変数の定義：

- (G) : Trace Graph
- (V) : Artifact Atom 集合
- (E) : Trace Link 集合

11.8 9.8 required link policy

policy では、Atom 種別ごとに必須リンクを定義する。

```
required_links:
  requirement:
    - type: refined_by
      target_kind: design
    min: 1
```

```
- type: verified_by
  target_kind: test
  min: 1
design:
- type: implemented_by
  target_kind: code
  min: 1
```

必須 link が欠落している場合、commit 不可である。

11.9 9.9 propagation policy

MVP では単純に、変更 Atom に直接接続された Atom へ required fire を立てる。

将来的には、変更種別ごとの policy を導入する。

```
propagation:
code_behavior_change:
  implemented_by:
    inverse: required
  verified_by:
    related: required
requirement_change:
  refined_by:
    forward: required
  verified_by:
    forward: required
```

12 10. Merge / Remote / Server 設計

12.1 10.1 merge の意味

Code Fire の merge は、source branch の sealed head を target branch の open directory へ統合し、必要な fire を立てる操作である。

merge 実行時点では正式履歴に commit を作らない。

```
merge
↓
target open directory が open-burning になる
↓
fire を解消する
↓
verify する
```

↓

commit する

12.2 10.2 merge 前提

source branch:

sealed head を持つ

open-burning ではない

target branch:

open されている

open-clean である

未完了変更がある target には merge できない。

12.3 10.3 merge inputs

B = common ancestor commit

S = source head commit

T = target head commit

W = target open directory

変数の定義：

- (B) : source と target の共通祖先 commit
- (S) : source branch の head commit
- (T) : target branch の head commit
- (W) : target branch の open directory

12.4 10.4 merge 処理

1. B, S, T の ContentManifest を読む
2. B, S, T の AtomIndex を読む
3. file-level 3-way merge を行う
4. Atom-level 変更を計算する
5. TraceGraph 更を merge する
6. conflict があれば conflict marker を出す
7. semantic conflict 候補を検出する
8. merge fire を作る
9. target を open-burning にする

12.5 10.5 server 方針

server は不整合状態を保存しない。

保存するもの：

sealed commit

sealed branch

merge request
review record
policy
permission

保存しないもの：

burning workspace
checkpoint
stash
unverified WIP state

12.6 10.6 remote clone

codefire clone cf://server/org/app/main main

remote branch を local branch として取得する。remote tracking branch は作らない。

最新版が必要な場合は、再度 clone する。

codefire clone cf://server/org/app/main main-latest

その後、必要なら merge する。

12.7 10.7 upload

codefire upload feature-login cf://server/alice/app/feature-login

upload できるのは sealed branch のみである。

server branch の現在 head を (R)、upload しようとする commit を (U) とする。

upload 可能条件：

[R ancestors(U)]

変数の定義：

- (R) : server 上の現在 branch head
- (U) : client が upload しようとしている commit
- (ancestors(U)) : commit (U) の祖先 commit 集合

つまり、fast-forward 可能な場合だけ upload できる。

12.8 10.8 merge request

codefire request-merge \
cf://server/alice/app/feature-login \
cf://server/org/app/main

merge request は、source sealed commit を target branch へ統合してほしいという申請である。

MR 作成時 target commit を (T_0)、現在 target head を (T_1) とする。

stale 件：

[T_0 T_1]

target が進んだら、MR は stale である。

server は分岐した commit を勝手に semantic merge しない。source 側で最新 target を clone し、local merge し、整合 commit を作り直す。

13 11. 内部アーキテクチャ設計

13.1 11.1 レイヤ構成

CLI Layer

ユーザコマンドを受ける

Command Service Layer

open, close, clone, scan, fire, extinguish, verify, commit, merge を実行する

Repository Store

sealed object, branch, policy, commit graph を保存する

Open Directory Manager

branch を実ディレクトリへ展開し、close/discard する

Indexer

ファイルから Artifact / Atom を抽出する

Trace Graph Engine

Atom 間の関係グラフを管理する

Fire Engine

変更に基づいて fire を生成・伝播・重複排除する

Policy Engine

commit 可能性、link 欠落、fire 未解消、stale resolution を判定する

Verifier

テスト、静的解析、生成物鮮度、trace completeness を検証する

Commit Sealer

consistent 状態を sealed commit として object store へ保存する

Merge Engine

sealed branch 同士を統合し、target open directory を burning にする

Remote Client / Server

clone, upload, merge request を扱う

13.2 11.2 Rust module 構成案

crates/

codefire-cli/
src/main.rs

cf-core/
src/types.rs
src/errors.rs

cf-store/
src/object_store.rs
src/commit_store.rs
src/branch_store.rs
src/hash.rs
src/canonical.rs

cf-workspace/
src/open.rs
src/close.rs
src/materialize.rs
src/discard.rs
src/identity.rs

cf-index/
src/indexer.rs
src/markdown.rs
src/python.rs
src/pytest.rs
src/atom_hash.rs

cf-trace/
src/graph.rs
src/links.rs
src/policy.rs

cf-fire/

src/fire_engine.rs
src/propagation.rs
src/resolution.rs
src/stale.rs

cf-policy/
src/commit_policy.rs
src/link_policy.rs
src/extinguish_policy.rs

cf-verify/
src/verifier.rs
src/external_command.rs
src/internal_checks.rs

cf-commit/
src/sealer.rs
src/certificate.rs

cf-merge/
src/merge_engine.rs
src/file_merge.rs
src/semantic_fire.rs

cf-remote/
src/client.rs
src/protocol.rs
src/upload.rs
src/clone.rs
src/merge_request.rs

13.3 11.3 scan 処理

1. open context 確認
2. codefire.yaml 読み込み
3. indexer 実行
4. current AtomIndex 生成
5. base AtomIndex と比較
6. changed_atoms 算出
7. TraceGraph 読み込み
8. FireEngine 実行
9. active fires 更新
10. obsolete fire 整理
11. status 更新

13.4 11.4 commit 処理

1. repo lock
2. branch lock
3. open identity 確認
4. copied directory でないことを確認
5. scan
6. verify
7. committable 判定
8. blob 保存
9. manifest 保存
10. atom index 保存
11. trace graph 保存
12. fire delta 保存
13. verification 保存
14. commit object 作成
15. branch head 更新
16. active state reset
17. open registry 更新

13.5 11.5 lock 設計

repo.lock

repository 全の構造更

branch-<name>.lock

branch head/open state 更

object-store.lock

object 書き込み

active-<open_instance_id>.lock

active state 更

branch head 更新は atomic rename で行う。

13.6 11.6 recovery

クラッシュ対応として codefire doctor を用意する。

orphan tmp 削除

object hash 再検証

branch head 参照検証

opened registry と codefire-open の整合確認

active state 破損確認

cache 再構築

cache は正式データではないため、壊れたら再構築する。

13.7 11.7 重要な実装判断

未 commit ファイル内容を repository に保存しない。

scan 時に作業中ファイルを blob 化すると、実質 checkpoint になる。したがって、BlobObject 保存は commit sealing 時だけ行う。

active state には fire/resolution/verification metadata を保存してよいが、それだけでは作業中ファイルを復元できないようにする。

14 12. MVP 実装計画

14.1 12.1 MVP の目的

MVP の目的は、次の中核価値を証明することである。

コードや仕様を変更すると関連成果物へ fire が立ち、fire が残っている状態では commit できず、すべて extinguish して verify に通った場合だけ sealed commit を作れる。

14.2 12.2 MVP 対象

Local repository
Local branch
open / close / clone
Markdown 仕様書
Markdown 設計書
Python コード
pytest テスト
Trace Link YAML
Fire ledger
Extinguish
Verify
Commit
Merge basic

14.3 12.3 MVP 対象外

Remote server
Merge request
AI 連携
多言語対応
OpenAPI

DB schema
高度な semantic merge
限管理
Review workflow
warning fire
GUI

14.4 12.4 Phase 1: Object store

作るもの：

object_id 計算
blob store
manifest object
commit object
branch object
init

成功条件：

repository を init できる
空の main branch を作れる
manifest を保存できる
commit object を保存できる
branch head を更新できる

14.5 12.5 Phase 2: open / close / clone

作るもの：

open
close
close --discard
clone
open registry
.codefire-open
copied directory 検出

成功条件：

main を open できる
main から feature を clone できる
feature を open できる
同一 branch 多重 open を拒否できる
close で実ディレクトリを消せる

14.6 12.6 Phase 3: Indexer

作るもの：

codefire.yaml parser
Markdown Atom 抽出
Python Atom 抽出
pytest Atom 抽出
AtomIndex 生成
duplicate atom id 検出

成功条件：

REQ/DES/CODE/TEST を抽出できる
Atom hash を計算できる
更 Atom を検出できる

14.7 12.7 Phase 4: Trace Graph

作るもの：

codefire.links.yaml parser
TraceGraphObject
link 参照検証
required link policy

成功条件：

REQ -> DES -> CODE
REQ -> TEST
の関係を読める
link 欠落で commit 拒否できる

14.8 12.8 Phase 5: Fire Engine

作るもの：

scan
changed atom detection
fire propagation
active fires
manual fire
obsolete fire
status 表示

成功条件：

コード変更で設計・仕様・テストに fire が立つ
fire が残ると commit 不可
revert すると auto fire が obsolete になる
manual fire は残る

14.9 12.9 Phase 6: Extinguish / stale

作るもの：

extinguish
resolution basis
no-change-required rationale 必須
stale resolution 判定

成功条件：

fire を extinguish できる
理由なし no-change-required は拒否
extinguish 後に対象 Atom を変えると stale になる
stale があると commit 不可

14.10 12.10 Phase 7: Verify / Commit

作るもの：

verify
external command runner
commit sealer
certificate
branch head 更新
active state reset

成功条件：

open fire なし
stale なし
link 欠落なし
pytest 成功
の場合だけ commit できる
commit 後 branch head が進む
open state が open-clean になる

14.11 12.11 Phase 8: Merge

作るもの：

common ancestor 探索
file-level 3-way merge

atom-level diff
merge fire
merge commit parents

成功条件：

feature を main に merge できる
merge 後 main が open-burning になる
fire 解消後に merge commit できる
merge commit parents が 2 つになる

14.12 12.12 MVP デモシナリオ

1. codefire init
 2. main を open
 3. サンプル仕様・設計・コード・テストを書く
 4. link を定義する
 5. verify して commit
 6. main から feature を clone
 7. feature を open
 8. コードを 30 分から 15 分へ変更
 9. scan で REQ/DES/TEST に fire が立つ
 10. commit が拒否される
 11. 仕様・設計・テストを更新する
 12. fire を extinguish する
 13. verify する
 14. commit できる
-

15 13. リスクと非目標

15.1 13.1 fire fatigue

fire が多すぎると、ユーザは fire を無視するようになる。

対策：

Atom 単位で管理する
ファイル単位 fire を避ける
変更種別を導入する
policy を調整可能にする
同種 fire をまとめる
AI に fire 解消案を出させる

15.2 13.2 commit が重くなりすぎる

Code Fire では commit 条件が厳しいため、大きな変更では commit まで時間がかかる。

対策：

整合可能な小さい変更単位に分割する
AI に「次の整合 commit」を作らせる
仕様・設計・コードを段階的に移行する

15.3 13.3 OS コピによる疑似 checkpoint

ユーザは open directory を OS レベルでコピーできる。

対策：

Code Fire としては管理しない
コピー先では Code Fire 操作を拒否する
open_instance_id で正規 open directory を識別する

15.4 13.4 Git 文化との摩擦

Git 利用者は以下ができないことに違和感を持つ可能性がある。

checkout できない
stash できない
WIP commit できない
partial commit できない
pull できない
fetch できない
rebase できない
force push できない

しかし、これらを入れると Code Fire の思想が崩れる。Code Fire は Git 互換の便利ツールではなく、開発規律を変えるツールである。

15.5 13.5 非目標

v0.2 時点の非目標：

Git 互換性
Git repository との共存
不整合状態の保存
checkpoint
stash
WIP commit
partial commit
staging area

pull/fetch/rebase

force upload

GUI

全言語対応

完全自動 semantic consistency 判定

AI による最終承認

15.6 13.6 課題

AI 連携

server-side verification certificate

commit signature

reviewer approval object

OpenAPI/DB schema support

semantic diff

large repository performance

policy version migration